

**LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK**

**PERTEMUAN 4
Mocking Object Using Mockito**



Disusun Oleh:

Nama : Abdullah Afif Habiburrohman
NIM : 537611/SV/24441
Kelas : BB / B2
Dosen : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

Contents

A Tujuan Pembelajaran	2
B Dasar Teori	2
C Hasil dan Pembahasan	2
C.1 Book.java	3
C.2 LibraryModel.java	3
C.3 NotificationService.java	3
C.4 LibraryControl	4
C.5 LibraryControlTest.java	4
C.6 Soal	7
C.7 Gambar Hasil Run	7
C.8 GitHub	8
D Kesimpulan	8
E Daftar Pustaka	9

A. Tujuan Pembelajaran

1. Mahasiswa mampu memahami cara kerja mocking object menggunakan Mockito
2. Mahasiswa mampu mengimplementasikan mocking object menggunakan Mockito

B. Dasar Teori

Mockito adalah sebuah framework mocking open-source yang sangat populer dan banyak digunakan untuk aplikasi berbasis Java. Framework ini memungkinkan pengembang perangkat lunak untuk membuat objek mock, yaitu objek tiruan yang meniru perilaku dari objek aslinya secara terkontrol. Dengan mengganti dependensi asli menggunakan mock, Mockito memungkinkan pengujian kode secara terisolasi tanpa perlu mengeksekusi metode aslinya secara langsung. Hal ini sangat membantu dalam menulis pengujian unit (unit test) yang bersih, mudah dipelihara, dan dapat diandalkan. Instalasi framework ini ke dalam proyek dapat dilakukan dengan menambahkan dependensi mockito-core melalui perangkat manajemen proyek seperti Apache Maven.

Terdapat beberapa konsep inti dalam penggunaan Mockito pada pengujian perangkat lunak:

1. Mock Objects (Objek Tiruan): Mock objects adalah implementasi palsu dari objek nyata yang menyimulasikan perilakunya untuk tujuan pengujian. Objek ini memungkinkan suatu kelas diuji secara terisolasi dengan mengganti dependensinya menjadi versi yang sepenuhnya dapat dikontrol dan diprediksi.
2. Stubbing: Stubbing adalah proses mendefinisikan apa yang harus dikembalikan oleh objek mock ketika sebuah metode spesifik dipanggil. Proses ini digunakan untuk menyimulasikan berbagai skenario, seperti mengembalikan nilai tetap, melempar eksepsi (exception), atau meniru respons sistem yang lambat. Implementasinya sering menggunakan sintaks `when(...).thenReturn(...)`.
3. Verification (Verifikasi): Verifikasi adalah proses untuk memeriksa apakah metode-metode tertentu benar-benar dipanggil pada objek mock selama eksekusi pengujian. Langkah ini memastikan bahwa kode yang diuji berinteraksi dengan dependensinya persis seperti yang diharapkan, contohnya memastikan sebuah metode dipanggil dengan argumen spesifik dalam jumlah waktu tertentu menggunakan fungsi `verify(...)`.

C. Hasil dan Pembahasan

Mahasiswa membuat proyek baru dengan isi kode tentang peminjaman buku dan mengimplementasikan mocking object. Framework yang digunakan untuk mocking

adalah Mockito. Berikut adalah kode-kode yang telah diimplementasikan untuk memenuhi tugas praktikum ini:

C.1 Book.java

```
1 package org.example;
2
3 public class Book {
4     private String id;
5     private String title;
6
7     public Book(String id, String title) {
8         this.id = id;
9         this.title = title;
10    }
11
12    public String getId() { return id; }
13    public String getTitle() { return title; }
14 }
```

Listing 1: Book.java

Kode diatas merupakan kelas Book yang memiliki dua atribut yaitu id dan title. Kelas ini juga memiliki konstruktor untuk menginisialisasi atribut-atribut tersebut serta getter untuk masing-masing atribut.

C.2 LibraryModel.java

```
1 package org.example;
2
3 import java.util.List;
4
5 public interface LibraryModel {
6     boolean isBookAvailable(String id);
7
8     void peminjaman(String title, String username);
9
10    List<Book> getAllBooks();
11 }
```

Listing 2: LibraryModel.java

Kode diatas merupakan interface LibraryModel yang memiliki tiga method yaitu isBookAvailable untuk mengecek ketersediaan buku berdasarkan id, peminjaman untuk melakukan peminjaman buku berdasarkan judul dan username, serta getAllBooks untuk mendapatkan daftar semua buku yang tersedia.

C.3 NotificationService.java

```
1 package org.example;
2
3 public interface NotificationService {
4     void sendNotification(String username, String title);
5 }
```

Listing 3: NotificationService.java

Kode diatas merupakan interface NotificationService yang memiliki satu method yaitu sendNotification untuk mengirimkan notifikasi kepada pengguna berdasarkan username dan judul buku yang dipinjam.

C.4 LibraryControl

```

1 package org.example;
2
3 public class LibraryControl {
4     private LibraryModel bookModel;
5     private NotificationService notificationService;
6
7     public LibraryControl(LibraryModel bookModel,
8 ↪ NotificationService notificationService) {
9         this.bookModel = bookModel;
10        this.notificationService = notificationService;
11    }
12
13    public boolean checkAvailability(String id) {
14        return bookModel.isBookAvailable(id);
15    }
16
17    public void borrowBook(String id, String title, String
18 ↪ username) {
19        bookModel.getAllBooks();
20
21        if (bookModel.isBookAvailable(id)) {
22            bookModel.peminjaman(title, username);
23            notificationService.sendNotification(username,
24 ↪ title);
25        }
26    }
27 }

```

Listing 4: LibraryControl.java

Kode diatas merupakan kelas LibraryControl yang memiliki dua atribut yaitu bookModel dan notificationService. Kelas ini juga memiliki konstruktor untuk menginisialisasi atribut-atribut tersebut. Kelas ini memiliki dua method yaitu check-Availability untuk mengecek ketersediaan buku berdasarkan id dan borrowBook untuk melakukan peminjaman buku berdasarkan id, judul, dan username. Pada method borrowBook, pertama-tama akan memanggil method getAllBooks untuk mendapatkan daftar semua buku yang tersedia. Kemudian akan mengecek ketersediaan buku menggunakan method isBookAvailable. Jika buku tersedia, maka akan melakukan peminjaman menggunakan method peminjaman dan mengirimkan notifikasi menggunakan method sendNotification.

C.5 LibraryControlTest.java

```

1 package org.example;
2
3 import org.junit.jupiter.api.BeforeEach;
4 import org.junit.jupiter.api.DisplayName;
5 import org.junit.jupiter.api.Test;
6 import org.mockito.InOrder;
7 import org.mockito.Mockito;
8
9 import static org.junit.jupiter.api.Assertions.assertFalse;
10 import static org.junit.jupiter.api.Assertions.assertTrue;
11 import static org.mockito.ArgumentMatchers.anyString;
12 import static org.mockito.Mockito.*;
13
14 class LibraryControlTest {
15
16     private LibraryModel bookModel;
17     private NotificationService notificationService;
18     private LibraryControl control;
19
20     @BeforeEach
21     void setUp() {
22         bookModel = Mockito.mock(LibraryModel.class);
23         notificationService = Mockito.mock(NotificationService.
↪ class);
24         control = new LibraryControl(bookModel,
↪ notificationService);
25     }
26
27     @Test
28     @DisplayName("Cek ketersediaan buku mengembalikan nilai
↪ dari model")
29     void testCheckAvailability() {
30         when(bookModel.isBookAvailable("B001")).thenReturn(true
↪ );
31         when(bookModel.isBookAvailable("B002")).thenReturn(
↪ false);
32
33         assertTrue(control.checkAvailability("B001"));
34         assertFalse(control.checkAvailability("B002"));
35     }
36
37     @Test
38     @DisplayName("Apabila buku tidak dipinjam (tersedia),
↪ fungsi peminjaman dipanggil")
39     void testBorrowBook_WhenAvailable_ShouldCallPeminjaman() {
40         when(bookModel.isBookAvailable("B001")).thenReturn(true
↪ );
41         control.borrowBook("B001", "Clean Code", "Afif");
42         verify(bookModel, times(1)).peminjaman("Clean Code", "
↪ Afif");
43     }
44
45     @Test
46     @DisplayName("Apabila buku sedang dipinjam (tidak tersedia)
↪ , fungsi peminjaman TIDAK dipanggil")

```

```

47     void
    ↪ testBorrowBook_WhenNotAvailable_ShouldNotCallPeminjaman()
    ↪ {
48         when(bookModel.isBookAvailable("B001")).thenReturn(
    ↪ false);
49         control.borrowBook("B001", "Clean Code", "Afif");
50         verify(bookModel, never()).peminjaman(anyString(),
    ↪ anyString());
51     }
52
53     @Test
54     @DisplayName("Verifikasi urutan eksekusi: getAllBooks ->
    ↪ peminjaman -> sendNotification")
55     void testBorrowBook_ExecutionOrder() {
56         when(bookModel.isBookAvailable("B001")).thenReturn(true
    ↪ );
57         control.borrowBook("B001", "Clean Code", "Afif");
58         InOrder inOrder = Mockito.inOrder(bookModel,
    ↪ notificationService);
59         inOrder.verify(bookModel).getAllBooks();
60         inOrder.verify(bookModel).peminjaman("Clean Code", "
    ↪ Afif");
61         inOrder.verify(notificationService).sendNotification("
    ↪ Afif", "Clean Code");
62     }
63 }

```

Listing 5: LibraryControlTest.java

Kode diatas adalah kode testing utama yang menggunakan Mockito. Terdapat `@BeforeEach` yang digunakan untuk menginisialisasi objek-objek yang akan digunakan dalam testing. Kemudian terdapat beberapa test yang akan dibahas satu persatu.

1. `testCheckAvailability`: Test ini digunakan untuk mengecek apakah fungsi `checkAvailability` mengembalikan nilai yang sesuai dengan yang diharapkan. Pada test ini, mahasiswa menggunakan `when` untuk menentukan nilai yang akan dikembalikan oleh method `isBookAvailable` pada `bookModel`. Kemudian mahasiswa menggunakan `assertTrue` dan `assertFalse` untuk memverifikasi hasil dari fungsi `checkAvailability`.
2. `testBorrowBook_WhenAvailable_ShouldCallPeminjaman` Test ini digunakan untuk mengecek apakah fungsi `peminjaman` dipanggil ketika buku tersedia. Pada test ini, mahasiswa menggunakan `when` untuk menentukan bahwa buku dengan id "B001" tersedia. Kemudian mahasiswa memanggil fungsi `borrowBook` dan menggunakan `verify` untuk memastikan bahwa fungsi `peminjaman` dipanggil dengan parameter yang sesuai.
3. `testBorrowBook_WhenNotAvailable_ShouldNotCallPeminjaman` Test ini digunakan untuk mengecek apakah fungsi `peminjaman` tidak dipanggil ketika buku tidak tersedia. Pada test ini, mahasiswa menggunakan `when` untuk menentukan bahwa buku dengan id "B001" tidak tersedia. Kemudian mahasiswa

memanggil fungsi borrowBook dan menggunakan verify untuk memastikan bahwa fungsi peminjaman tidak dipanggil.

4. testBorrowBook_ExecutionOrder Test ini digunakan untuk memverifikasi urutan eksekusi fungsi-fungsi yang dipanggil dalam fungsi borrowBook. Pada test ini, mahasiswa menggunakan InOrder untuk memverifikasi urutan eksekusi fungsi-fungsi yang dipanggil. mahasiswa memastikan bahwa fungsi getAllBooks dipanggil terlebih dahulu, kemudian fungsi peminjaman, dan terakhir fungsi sendNotification.

C.6 Soal

Implementasikan mockito, identifikasi stub dan mock yang bisa digunakan dalam kasus berikut. Silahkan menambahkan fungsi atau class yang dibutuhkan.

1. Tambahkan fungsi untuk cek ketersediaan buku pada controller dan model.. Cek ketersediaan buku berdasarkan id. Buatlah test method untuk method cek ketersediaan pada controller. Terjawab pada C.2 dan 1.
2. Meminjam buku (id buku). Buat 2 test scenario:
 - (a) Apabila tidak sedang dipinjam maka fungsi peminjaman buku akan dipanggil. Terjawab pada 2.
 - (b) Apabila buku sedang dipinjam, lakukan verifikasi bahwa fungsi peminjaman buku tidak dipanggil. Terjawab pada 3.
3. Tambahkan class model notificationService yang akan mengirimkan notifikasi apabila buku berhasil dipinjam atau dikembalikan. Urutan proses yang diharapkan (eksplor penggunaan fungsi InOrder untuk memastikan urutan pemanggilannya sesuai):
 - (a) getAllBooks()
 - (b) Peminjaman(title, username)
 - (c) sendNotification(username, title)

Terjawab pada C.3 dan 4.

C.7 Gambar Hasil Run

Berikut adalah gambar hasil run dari kode diatas:

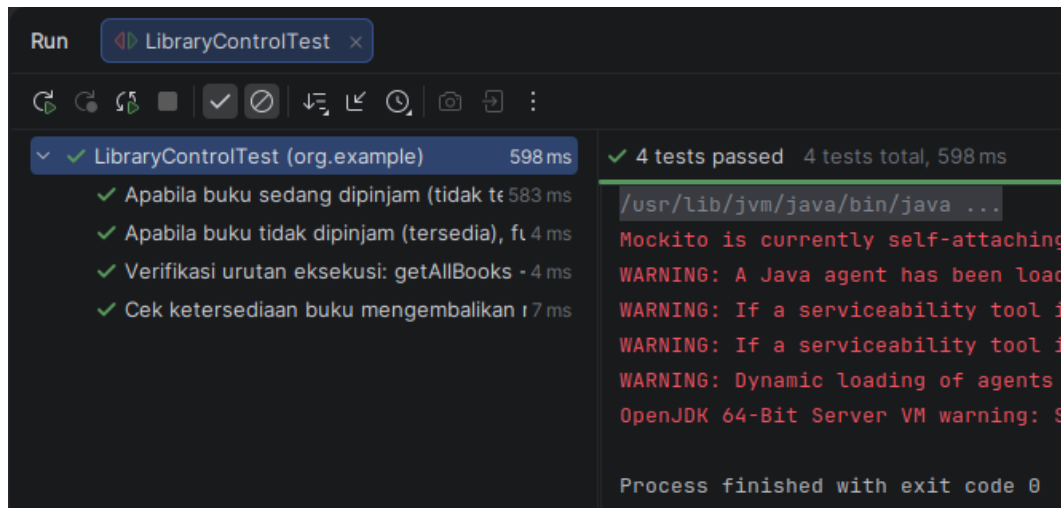


Figure 1: Hasil Run

Warning yang terdapat pada gambar tersebut tidak mempengaruhi hasil run. Error tersebut muncul karena mockito melakukan injeksi kode selama proses mocking, sehingga kode yang dihasilkan tidak dapat dikenali oleh IDE. Namun, semua test berhasil dijalankan dengan hasil yang sesuai dengan yang diharapkan.

C.8 GitHub

Berikut adalah link repository di GitHub yang berisi kode diatas <https://github.com/bukunya/PPPL/tree/Pertemuan4>

D. Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Framework Mockito terbukti sangat efektif dalam melakukan pengujian unit (unit testing) dengan mengisolasi kelas yang diuji (LibraryControl) dari antarmuka dependensinya (LibraryModel dan NotificationService). Hal ini membuat pengujian menjadi lebih fokus pada logika bisnis tanpa bergantung pada implementasi infrastruktur asli.
2. Penggunaan teknik stubbing melalui metode `when(...).thenReturn(...)` memungkinkan penguji untuk merekayasa skenario pengujian dengan mudah, seperti mensimulasikan kondisi di mana buku tersedia atau tidak tersedia, tanpa memerlukan modifikasi pada basis data nyata.
3. Verifikasi interaksi (behavior verification) berhasil diimplementasikan menggunakan fungsi `verify()`. Fungsi ini memastikan bahwa metode tertentu dipanggil sesuai skenario, baik dipanggil tepat satu kali maupun tidak dipanggil sama sekali (`never()`).

4. Fitur InOrder pada Mockito terbukti sangat handal untuk memastikan kebenaran alur program secara ketat dengan memvalidasi urutan eksekusi pemanggilan metode. Pada kasus ini, urutan proses sistem yaitu `getAllBooks()` → `peminjaman()` → `sendNotification()` berhasil diverifikasi dengan tepat.

E. Daftar Pustaka

- [1] Maven - Mockito mock objects library core API and implementation <https://mvnrepository.com/artifact/org.mockito/mockito-core/5.20.0>
- [2] GeeksforGeeks - Introduction to Mockito <https://www.geeksforgeeks.org/java/introduction-to-mockito/>